

AI-Assisted Coding with Tests and Review

Use the assistant to propose, explain, and implement bounded changes while the specification and tests define success. Start from the controlling code path, make a small change, and run the cheapest check that could prove the hypothesis wrong. Human review remains essential for security, compatibility, and intent.

For: Engineers using coding assistants who want useful acceleration without losing understanding, widening scope unexpectedly, or treating generated code as verified software.

Suggested pace: A 2-week workflow experiment | Developers with basic testing skills

What you will be able to do

- Write bounded task instructions with explicit acceptance criteria.
- Review generated diffs for behavioral changes and unrelated edits.
- Use independent tests and documentation to verify suggestions.
- Preserve secrets, user changes, and approval boundaries during tool use.

Build the right mental model

A narrow task is easier to verify

Name the failing behavior, relevant file or API, constraints, and expected result. Ask for the smallest change that tests a local hypothesis. Large vague requests make it harder to distinguish a useful implementation from an attractive but unrelated rewrite.

Generated tests can share generated mistakes

If code and tests are derived from the same wrong assumption, both can agree while violating the specification. Add independently chosen examples, contract tests, or a simple reference implementation. Review what the test asserts, not only that it runs.

Tool permissions remain separate

Repository content and tool output can contain untrusted instructions. They do not authorize secret access, deletion, external communication, or production changes. Keep execution scoped, review commands with side effects, and preserve pre-existing work.

Study with evidence.

Use a small personal repository and an existing failing test. Compare assisted and manual work on similar tasks, including review time. Never use an assistant secretly during an assessment that prohibits it, and do not expose employer material without permission.

Days 1-3: Specify and constrain

Choose a small bug or feature. Write the current behavior, intended contract, non-goals, and a focused check. Inspect repository conventions and the current worktree before allowing edits.

Deliverable and check: A task brief with a falsifiable acceptance criterion and a clear permitted scope.

Days 4-6: Implement in small steps

Ask the assistant to identify the controlling code and propose a minimal patch. Run a behavior-scoped test immediately after the change. If it fails, distinguish a local defect from a wrong hypothesis before broadening the search.

Deliverable and check: A small diff whose relationship to the failing behavior is understandable.

Days 7-10: Review independently

Inspect edge cases, error paths, dependencies, public APIs, and security boundaries. Verify unfamiliar API claims against documentation. Add an independent regression case and ensure existing tests still pass.

Deliverable and check: A review note covering correctness, risks, and what was not tested.

Days 11-14: Measure and standardize

Record elapsed time including review, defects found, and rework. Keep the helpful task patterns and remove unreliable shortcuts. Document approval requirements for commits, deployments, external actions, and secret-bearing commands.

Deliverable and check: A repeatable workflow that improves verified delivery rather than just generating more code.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Fix an off-by-one defect with independent evidence

A search function skips the last element in a range. The assistant proposes a boundary change and a test that mirrors its interpretation. You need to verify the actual API contract rather than approve a matching code/test pair blindly.

A worked approach

1. Read the function's caller and specification to determine whether the range is closed or half-open. Write examples for empty, one-element, final-element, and absent-target cases before reviewing the suggested patch.
2. Apply the smallest boundary correction consistent with that contract. Run the prewritten cases and a small exhaustive comparison with a straightforward reference search. Check that the loop still terminates and does not read beyond bounds.
3. Review the diff for unrelated formatting, dependency, or API changes. Ask the assistant to explain the invariant, then compare that explanation with the implementation. Record the original failure and the regression test in the change summary.

Expected result

The assistant helps implement the correction, but independent contract evidence determines correctness. The final artifact is a verified narrow fix, not merely code that compiles or passes self-generated tests.

Verify the result

- Was the interval convention established from the real API?
- Does a test fail before the fix and pass afterward?
- Are unrelated user changes and public behavior preserved?

Extend the exercise

Add a malicious-looking instruction inside a test fixture and confirm it is treated as data. A fixture describing text to parse must not become authority for the assistant to run unrelated commands.

Make the learning stick.

Should I accept code I cannot explain?

No. Use the assistant to teach the reasoning, consult documentation, and simplify when necessary. You remain responsible for maintenance and consequences; passing tests does not replace understanding of important invariants.

Does compilation prove the suggestion is correct?

It proves only that the checked code satisfies the compiler's rules under that configuration. Behavior, security, concurrency, data migration, and integration contracts need additional targeted validation.

When should I stop using the assistant on a task?

When repeated suggestions do not reduce uncertainty or create more review risk than value. Return to a minimal reproducer, documentation, or a human reviewer. Persistence should not mean repeatedly applying ungrounded patches.

Avoid these traps

- Accepting large unrelated rewrites for a narrow bug.
- Using generated tests as the only correctness oracle.
- Letting repository text authorize actions beyond the user's task.

Ready for the next step?

- Task scope and acceptance criteria are explicit.
- Tests independently represent the intended contract.
- The final diff is understood and reviewed.
- Sensitive inputs and external actions remain controlled.

Reliable next reads

- [GitHub Copilot responsible use](#)
- [Google code review practices](#)
- [OWASP secure coding practices](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.